Data Structures And Algorithms
-0094224-

- **Student 1:** Suheil Khourieh – [redacted]
- **Student 2:** [redacted] – [redacted]

Question 1:

Prove/Disprove the following claims using the definition. You may use everything that was taught in the lectures and tutorials, but you may not use the definition using limits. In this question, the domain of the functions is $\mathbb{Z}_{\geq 0}$, the range is $\mathbb{R}$ and you may assume that the functions are asymptotically positive.

Part 1:

For every $f(n), g(n)$ as defined in the question, if $f(n) \in O(g(n)$ then $g(n) \in \Omega\big(f(n)\big)$.

Solution:

The claim is correct!

We are asked to prove the following:

$$f(n) \in O\big(g(n)\big) \Rightarrow g(n) \in \Omega\big(f(n)\big)$$

By definition:

$$\exists c_1, n_1 > 0 : 0 \leq f(n) \leq c_1 g(n), \forall n \geq n_1$$
$$\Rightarrow \exists c_2, n_2 : 0 \leq c_2 f(n) \leq g(n), \forall n \geq n_2$$

We are given that $f(n) \in O\big(g(n)\big)$, thus by definition there exists $c_1, n_1 > 0$ that hold the first part of the claim, let's multiply all sides with $\frac{1}{c_1}$ in the first claim, we get the following: $\exists c_1, n_1 > 0 : 0 \leq \frac{1}{c_1} f(n) \leq g(n), \forall n \geq n_1$, by choosing $c_2 = \frac{1}{c_1}$ and $n_2 = n_1$ we get the following $\exists c_2 = \frac{1}{c_1}, n_2 = n_2 : 0 \leq c_2 f(n) \leq g(n), \forall n \geq n_2$.

This is exactly what we are trying to prove. ■

Part 2:

For every $f(n), g(n)$ as defined in the question, if $f(n) \in o\big(g(n)\big)$ then $g(n) \notin o\big(f(n)\big)$.

Solution:

The claim is correct!

Let's assume by contradiction that the claim is incorrect, in other words, let's assume that the following claim is correct: $f(n) \in o\big(g(n)\big) \Rightarrow g(n) \in o\big(f(n)\big)$.

By definition, we know that $\forall c_1 > 0\ \exists n_1 > 0 : 0 \leq f(n) < c_1 g(n), \forall n \geq n_1$, we will try to prove that this implies $g(n) \in o\big(f(n)\big)$ by definition:

$\forall c_2 > 0, \exists n_2 > 0 : 0 \leq g(n) < c_2 f(n), \forall n \geq n_2$ and we reach a contradiction.

The second claim should hold for every $c_2 > 0$, let $c_2 = \frac{1}{c_1} > 0$, we get that we are trying to prove $\exists n_2 > 0 : 0 \leq g(n) < \frac{1}{c_1} f(n), \forall n \geq n_2$, let's multiply all sides with c_1 we get the following: $\exists n_2 > 0 : 0 \leq c_1 g(n) < f(n), \forall n \geq n_2$, there can't be any $c_2 > 0$ that holds the claim for every $n \geq n_2$ because we know from the first part of the claim that there exists n_1 that hold the this claim $0 \leq f(n) < c_1 g(n)$ for every $n \geq n_1$.

$(\rightarrow\leftarrow)$, the claim in the question is indeed correct. ■

Part 3:

For every $f(n), g(n)$ as defined in the question, the following claim is true:

$$f(n) \in o\big(g(n)\big) \; or \; g(n) \in o\big(f(n)\big)$$

Solution:

The claim is incorrect!

Disprove the claim using a counter-example!

Let $f(n) = g(n) = any$ asymptotically positive function!

We need to disprove that neither of the following claims is correct:

$$(1)\, \forall c_1 > 0, \exists n_1 > 0 : 0 \le f(n) < c_1 g(n), \forall n \ge n_1$$
$$(2)\, \forall c_2 > 0, \exists n_2 \ge 0 : 0 \le g(n) < c_2 f(n), \forall n \ge n_2$$

Because both claims should hold for every $c_1, c_2 > 0$, let $c_1 = c_2 = 1$ and we will see that for those values both claims cannot hold:

According to claim (1), when $c_1 = 1$ there exists $n_1 > 0$ that holds the following: $0 \le f(n) < 1 \cdot g(n), \forall n \ge n_1$ but we choose $f(n) = g(n)$ thus we get that $0 \le f(n) < f(n)$ which is not possible.

In a similar way, according to claim (2), when $c_2 = 1$ there exists $n_2 > 0$ that holds the following: $0 \le g(n) < 1 \cdot f(n), \forall n \ge n_2$ but we choose $f(n) = g(n)$ thus we get that $0 \le g(n) < g(n)$ which is not possible.

This is an example of two function that disprove the claim given in the question.

$\Rightarrow$ *The claim is incorrect.*

Part 4:

For every $f(n), g(n)$ as defined in the question, the following is true:

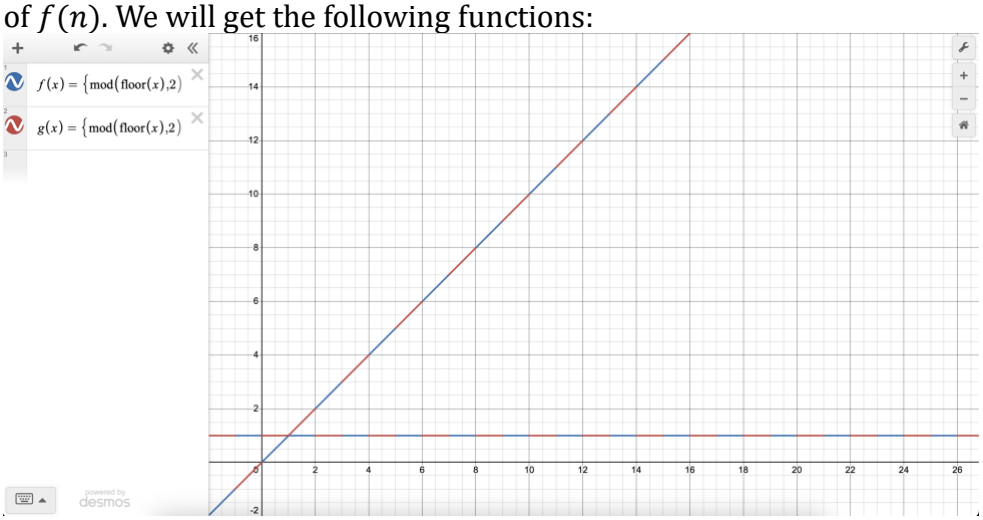
$$f(n) \in O\big(g(n)\big) \; or \; g(n) \in O\big(f(n)\big)$$

Solution:

The claim is incorrect!

Disprove the claim using a counter example!

Let $f(n) = \begin{cases} n, if \; \lfloor n \rfloor \% 2 = 0 \\ 1, if \; \lfloor n \rfloor \% 2 = 1 \end{cases}$, $g(n) = \begin{cases} 1, if \; \lfloor n \rfloor \% 2 = 0 \\ n, if \; \lfloor n \rfloor \% 2 = 1 \end{cases}$, in other words, let $f(n)$ be n when the floor value of n is even and let it be 1 other wise, $g(n)$ is the exact opposite of $f(n)$. We will get the following functions:



We can easily see that depending on the floor value of n we get different results, further more, we can split the problem to two cases:

Case 1 is when the floor value of n is even, we get that $g(n) \le f(n)$, even more than

that we can see that $g(n) \in O(f(n))$ if and only if the floor value of n is even.
Case 2 is when the floor value of n is odd, we get that $f(n) \leq g(n)$, we also get that $f(n) \in O(g(n))$ if and only if the floor value of n is odd.
To further explain our counter example, depending on the floor value of n, sometimes $f(n) \in O(g(n))$ and sometimes $g(n) \in O(f(n))$ but it is not consistant, thus the claim is incorrect.

Question 2:

In this question, we will define new types of relations between functions. In a simillar way to the tutorials, the domain of the functions is $\mathbb{Z}_{\geq 0}$, the range is $\mathbb{R}$ and you may assume that the functions are asymptotically positive. Let $f(n) \in X\big(g(n)\big)$ if there exists $c > 0, n_0$ where for every $n > n_0$ the following takes place: $g(n) - c \leq f(n) \leq g(n)$. In addition, let $f(n) \in Y\big(g(n)\big)$ if there exists $c > 0, n_0$ where for every $n > n_0$ the following takes place: $g(n) \leq f(n) \leq g(n) + c$.

In this question, you may not use definitions that are based on limits, and assume that $f(n) \neq g(n)$ and that $g(n)$ is monotonically increasing.

Prove\Disprove:

Part 1:

$f(n) \in X\big(g(n)\big) \Rightarrow f(n) \in \Omega\big(\mathrm{g(n)}\big)$

Solution:

The claim is incorrect!

Disprove the claim using a counter example:

Let $f(n) = \frac{1}{n}, g(n) = 1$.

We notice that $f(n) \in X\big(g(n)\big)$, we can prove this simply by choosing $c = n_0 = 1$, we get: $g(n) - 1 \leq f(n) \leq g(n), \forall n \geq 1 \Rightarrow 0 \leq \frac{1}{n} \leq 1, \forall n \geq 1$.

Part 2:

$f(n) \in \Omega\big(g(n)\big) \Rightarrow f(n) \in X\big(g(n)\big)$

Solution:

The claim is incorrect!

Disprove the claim using a counter example:

Let $f(n) = n + 1, g(n) = n$, we can easily see that $f(n) \in \Omega\big(g(n)\big)$, for example for $c_1 = \frac{1}{2}, n_1 = 1$ we get the following:

$0 \leq c_1 \cdot n \leq n + 1, \forall n \geq n_1 \Rightarrow 0 \leq \frac{1}{2} n \leq n + 1, \forall n \geq 1$, on the other hand, $f(n) \notin X\big(g(n)\big)$ simply because according to the definition of $f(n) \in X\big(g(n)\big)$, there exists n_2, where for every $n > n_2$ the following must hold $f(n) \leq g(n)$, but we can easily see the there doesn't exist any n_2 where for every $n > n_2$ the following holds $n + 1 \leq n$, simply because by subtracting n from the equation we get $1 \leq 0$ ($\rightarrow\leftarrow$).

Thus the claim is incorrect, disproven using a counter example! ■.

Part 3:

$f(n) \in Y(g(n)) \Rightarrow f(n) \in O(g(n))$

Solution:

The claim is correct!

We know that $f(n) \in Y(g(n)) \Leftrightarrow \exists c_1 > 0, n_0 : g(n) \leq f(n) \leq g(n) + c_1, \forall n \geq n_0$.
We know that $g(n)$ is monotonically increasing and that c_1 is positive, thus the following must be true: $\exists a > 0, n_1 : g(n) + c_1 = a \cdot g(n)$.
From the definition of $f(n) \in Y(g(n))$ we get:
$\exists c_1 > 0, n_0 : g(n) \leq f(n) \leq g(n) + c_1, \forall n \geq n_0$, we can replace $g(n) + c_1$ with $a \cdot g(n)$ starting from n_1 and get that $g(n) \leq f(n) \leq a \cdot g(n), \forall n \geq \max\{n_0, n_1\}$.
$\Rightarrow$By choosing $c = a \; and \; n_2 = \max\{n_0, n_1\}$ we get:
$\exists c > 0, n_2 : 0 \leq f(n) \leq c \cdot g(n), \forall n \geq n_2$. ■

Part 4:

$f(n) \in O(g(n)) \Rightarrow f(n) \in Y(g(n))$

Solution:

The claim is incorrect!

Disprove the claim using a counter example:
Let $f(n) = n, g(n) = n + 1$.
We can easily see that $f(n) \in O(g(n))$, for example, let $c_1 = 1, n_1 = 1$ we get that for every $n \geq n_1 = 1$ the following is true: $0 \leq n \leq n + 1, \Rightarrow f(n) \in O(g(n))$.
But, $f(n) \notin Y(g(n))$ because it does not hold the first part of the definition:
$g(n) \leq f(n)$ that is in our case $n + 1 \leq n$. ■

Question 3:

In this question, the function T is the time complexity function. All the functions in this question are asymptotically positive.
Write true/false and explain.

Part 1:

Given any algorithm A. If for every constant $c > 0$ there exists $n_0 > 0$ where there exists an infinite seires of instances $\{I_n\}_{n \geq n_0}$ where for every instance in the series with size n it holds $T_A(I) < cg(n)$ then $T_A(n) = o\big(g(n)\big)$.

Solution:

The claim is incorrect!
Given: $\forall\, I : T_A(I) < c \cdot g(n)$
Disprove the claim using a counter example:
Let:
$g(n) = 2n, \max T_A(I) = \frac{n}{4}, c = 1$
$\Rightarrow \frac{n}{4} < 2n \Rightarrow \max T_A(I) < c \cdot g(n)$
We have learned that:
$\lim\limits_{n \to \infty} \frac{\boldsymbol{max\, T_A(I)}}{g(n)} = 0 \;\Leftrightarrow \boldsymbol{max\, T_A(I) = o(g(n))}$
But if we calculate the following limit:
$\lim\limits_{n \to \infty} \frac{g(n)}{\boldsymbol{max\, T_A(I)}} = \lim\limits_{n \to \infty} \frac{2n}{\frac{n}{4}} = 8 \neq 0$

Part 2:

Given any algorithm A. If there exists constants $c_1, c_2 > 0$ and there exists an infinite series of instances $\{I_n\}_{n \geq 1}$ where for every instance in the series with size n it holds that $c_1 g(n) \leq T_A(I) \leq c_2 g(n)$ then $T_A(n) = \Theta\big(g(n)\big)$

Solution:

The claim is correct!
Given:
$\forall I : c_1 \cdot g(n) \leq T_A(I) \leq c_2 \cdot g(n)$
We can infer:
$\forall I : c_1 \cdot g(n) \leq T_A(I) \leq c_2 \cdot g(n) \Rightarrow \forall I :\; T_A(I) = \Theta\big(g(n)\big)$
Since $T_A(I)$ (for each I) is always founded by the range of the two bounds (the lower bound $(c_1 \cdot g(n))$ and the upper bound $(c_2 \cdot g(n))$):
$c_1 \cdot g(n) \leq (\max T_A(I)) \leq c_2 \cdot g(n) \Rightarrow (\max T_A(I)) = \Theta\big(g(n)\big)$
So:
$T_A(n) = \max T_A(I) = \Theta\big(g(n)\big) \Rightarrow T_A(n) = \Theta\big(g(n)\big)$. ■

Part 3:

Given any algorithm A. If for every instance I where $|I| = n$ it holds $g(n) \leq T_A(I)$ then $T_A(n) = \Omega\big(g(n)\big)$.

Solution:

The claim is correct!

$\forall I : g(n) \leq T_A(I)$

We can infer:

$\forall I : \ g(n) \leq T_A(I) \Rightarrow \forall I : T_A(I) = \Omega\big(\mathrm{g(n)}\big)$

$\Rightarrow (\max T_A(n)) = \Omega\big(g(n)\big)$

We have learned that:

$T_A(n) = \max T_A(I)$

So:

$T_A(n) = \max T_A(I) = \Omega\big(g(n)\big) \Rightarrow T_A(n) = \Omega\big(g(n)\big)$

Part 4:

Given any algorithm A. If $T_A(n) = \Omega(2^n)$, then there exists $c, n_0 > 0$ where for every instance I with size $n \geq n_0$ it holds $c2^n \leq T_A(I)$.

Solution:

The claim is incorrect!

Given:

$\forall I : c \cdot 2^n \leq T_A(I)$

The claim assumes that every instance I has a runtime at least 2^n. $T_A(n)$ only represents the maximum runtime, not necessarily applicable to all instancases, so not every I may reach this lower bound.

Question 4:

In this question the input for the algorithm is two arrays A and B with the same size. Calculate a tight asymptotic barrier, Θ. You may not use sum.

```
Alg_4(A, B)
1.  n = A.length
2.  if n < 10^6 then
3.      print SMALL
4.  else
5.      print B[1]
6.      sum = 0
7.      for i = 1 to 10^6 do
8.          sum = sum + A[i] + B[i]
9.      if sum < 10^20 then
10.         print sum
11.     else
12.         for i = 1 to n do
13.             for j = 1 to n do
14.                 A[i] = A[i] + B[i]
```

Solution:

We can easily see that lines 1-6 take $\Theta(1)$, the loop line 7-8 also takes $\Theta(1)$ because it runs fixed times, lines 9-11 take $\Theta(1)$ also. All of these operations run a fixed amount of times, thus for now it is $\Theta(1)$.

The loop line 12 runs n times where n is the size of the arrays A, B.

At each iteration it runs another loop (line 13) that also runs n times.

At each iteration of the inner loop, a fixed number of operations is done, in total we have:

Every inner iteration has $\Theta(1)$, we have n iterations so in total $\Theta(n)$.

The outer loop preforms n iterations where each iteration runs n operations.

In total we have $\Theta(n^2)$.

The tight asymptotic barrier of the algorithm $Alg_4(A, B)$ is $\Theta(n^2)$.

Question 5:

Calculate tight asymptotic barrier Θ, without using sum, for the algorithm Alg_5.

```
Alg_5(n)
1. for i = 1 to n do
2.   if i % 2 = 0 then
3.     for j = 1 to i do
4.       print(j)
5.   if i % 3 = 0 then
6.     for j = 1 to i/3 do
7.       print(j)
```

Solution:

The loop line 1 does n iterations, we can see that the value of n **is not** manipulated anywhere inside the loop and there is no break operation inside the loop, thus the loop will indeed complete all n iterations.

Depending on the value of i (The index of the outer loop), we might enter loop line 2 or loop line 5 or nither of them, it really doesn't matter.

Loop line 2 iterates i times, while loop line 5 iterates $\frac{i}{3}$ times, both can be looked at like $\Theta(i)$. When getting close to finishing the outer loop, the value of i will get close to n and it will enter loop at line 2 at $i = n$ or $i = n - 1$. It doesn't really make a difference, we get that we do $\Theta(n^2)$ operations, thus the tight asymptotic barrier for Alg_5 is $\Theta(n^6)$.